Lecture 8. Factors, Dates and Times

forcats, lubridate: creating, components, arithmetic, time zones

Contents

8.1 Factors **2**
- 8.1.1 Creating factors . . . 2
- 8.1.2 The General Social Survey . . . 2
- 8.1.3 Modifying factor order . . . 2
- 8.1.4 Modifying factor levels . . . 3

8.2 Dates and times **4**
- 8.2.1 Creating dates and date-times . . . 4
- 8.2.2 Date-time components . . . 5
- 8.2.3 Time spans . . . 6
- 8.2.4 Time zones . . . 7

8.3 Selected exercises **7**

8.1 Factors

Motivation. Factors give a fixed set of values (typos caught) and a non-alphabetical order; forcats manipulates them.

8.1.1 Creating factors

Months as strings: typos go undetected, and they sort alphabetically.

```
x1 <- c("Dec", "Apr", "Jan", "Mar")
x2 <- c("Dec", "Apr", "Jam", "Mar")
sort(x1)
```

```
## [1] "Apr" "Dec" "Jan" "Mar"
```

```
month_levels <- c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")
y1 <- factor(x1, levels = month_levels)
sort(y1)
```

```
## [1] Jan Mar Apr Dec
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

```
y2 <- factor(x2, levels = month_levels)
y2
```

```
## [1] Dec  Apr  <NA> Mar
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

Values outside the **levels** silently become `NA` (`readr::parse_factor()` warns). Default levels: sorted unique values; `factor(x1, levels = unique(x1))` keeps first appearance. `levels(y1)` lists them.

8.1.2 The General Social Survey

`forcats::gss_cat`: a sample of the US General Social Survey (marital status, race, income, party, religion). See levels with `count()` or a bar chart; `scale_x_discrete(drop = FALSE)` shows empty levels.

```
gss_cat %>% count(race)
```

```
## # A tibble: 3 x 2
##   race      n
##   <fct> <int>
## 1 Other  1959
## 2 Black  3129
## 3 White 16395
```

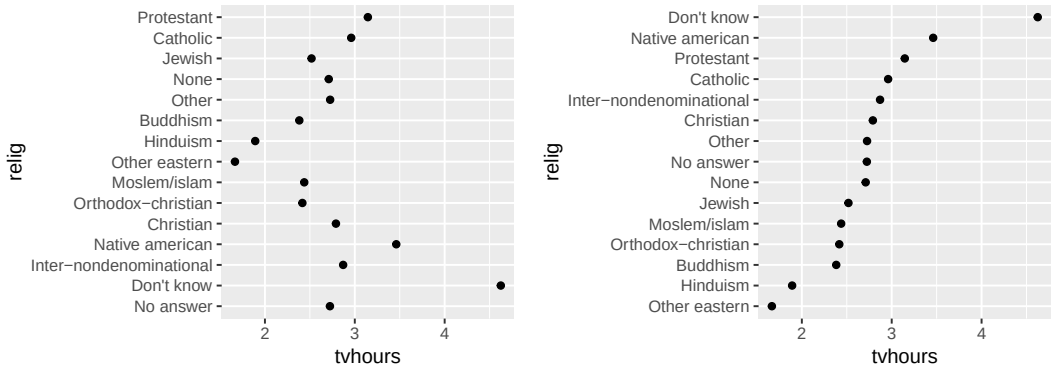
Two operations: change the *order* or the *values* of the levels.

8.1.3 Modifying factor order

`fct_reorder(f, x)` orders levels by the median of `x` (or `.fun`); TV hours by religion:

```
relig_summary <- gss_cat %>%
  group_by(relig) %>%
  summarise(age = mean(age, na.rm = TRUE),
            tvhours = mean(tvhours, na.rm = TRUE), n = n())
ggplot(relig_summary, aes(tvhours, relig)) + geom_point()
relig_summary %>%
```

```
  mutate(relig = fct_reorder(relig, tvhours)) %>%
  ggplot(aes(tvhours, relig)) + geom_point()
```



Reorder in `mutate()`, not `aes()`. Keep natural orders: `fct_reorder(rincome, age)` would scramble income brackets; `fct_relevel(rincome, "Not applicable")` only moves that level to the front.

`fct_reorder2(f, x, y)`: by `y` at the largest `x` (lines match the legend); `fct_infreq(f)`: by frequency (bar charts; `fct_rev()` reverses).

8.1.4 Modifying factor levels

`fct_recode()` renames (new on the left), leaves others; several old to one new combines:

```
gss_cat %>%
  mutate(partyid = fct_recode(partyid,
    "Republican, strong"    = "Strong republican",
    "Republican, weak"      = "Not str republican",
    "Independent, near rep" = "Ind,near rep",
    "Independent, near dem" = "Ind,near dem",
    "Democrat, weak"        = "Not str democrat",
    "Democrat, strong"      = "Strong democrat",
    "Other"                 = "No answer",
    "Other"                 = "Don't know",
    "Other"                 = "Other party"
  )) %>%
  count(partyid)
```

```
## # A tibble: 8 x 2
##   partyid                 n
##   <fct>               <int>
## 1 Other                 548
## 2 Republican, strong   2314
## 3 Republican, weak     3032
## 4 Independent, near rep 1791
## 5 Independent          4119
## # i 3 more rows
```

Combine with care (different groups merge). `fct_collapse()`: a vector of old levels per new one:

```
gss_cat %>%
  mutate(partyid = fct_collapse(partyid,
    other = c("No answer", "Don't know", "Other party"),
    rep = c("Strong republican", "Not str republican"),
    ind = c("Ind,near rep", "Independent", "Ind,near dem"),
    dem = c("Not str democrat", "Strong democrat"))) %>%
  count(partyid)
```

```
## # A tibble: 4 x 2
##   partyid     n
##   <fct>   <int>
## 1 other     548
## 2 rep      5346
## 3 ind      8409
## 4 dem      7180
```

`fct_lump(f, n = 10)`: the `n` most frequent levels, the rest "Other".

8.2 Dates and times

Motivation. Unequal years, 23- or 25-hour days and time zones: lubridate keeps them apart.

`lubridate` (older tidyverse: load separately) has **date** (`<date>`), **time** (`<time>`, `hms`) and **date-time** (`<dttm>`, POSIXct), an instant. Use the simplest type (date-times bring time zones). `today()`, `now()`.

8.2.1 Creating dates and date-times

From strings. Name the order of `y`, `m`, `d`; add `_h`, `_hm`, `_hms` for times. Unquoted numbers work too.

```
ymd("2017-01-31")
```

```
## [1] "2017-01-31"
```

```
mdy("January 31st, 2017")
```

```
## [1] "2017-01-31"
```

```
dmy("31-Jan-2017")
```

```
## [1] "2017-01-31"
```

```
ymd_hms("2017-01-31 20:11:59")
```

```
## [1] "2017-01-31 20:11:59 UTC"
```

```
mdy_hm("01/31/2017 08:01")
```

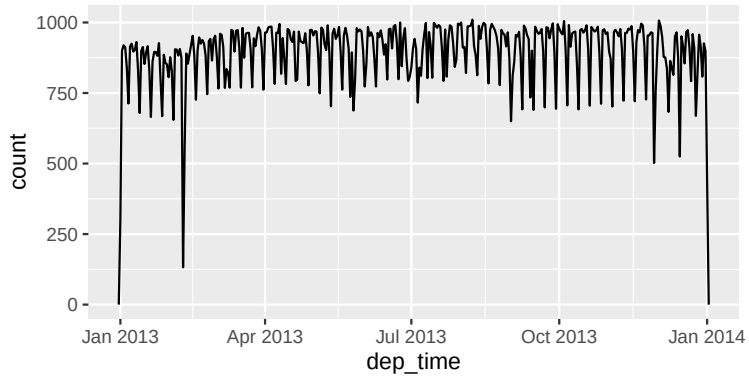
```
## [1] "2017-01-31 08:01:00 UTC"
```

From components. `make_date()`, `make_datetime()`; `HHMM` times need a helper:

```
make_datetime_100 <- function(year, month, day, time) {
  make_datetime(year, month, day, time %/% 100, time %% 100)
}
flights_dt <- flights %>%
  filter(!is.na(dep_time), !is.na(arr_time)) %>%
  mutate(
    dep_time = make_datetime_100(year, month, day, dep_time),
    arr_time = make_datetime_100(year, month, day, arr_time),
    sched_dep_time = make_datetime_100(year, month, day, sched_dep_time),
    sched_arr_time = make_datetime_100(year, month, day, sched_arr_time)
  ) %>%
  select(origin, dest, ends_with("delay"), ends_with("time"))
```

Numerically a date-time is seconds (a date days): binwidth 86400 is one day:

```
flights_dt %>%
  ggplot(aes(dep_time)) +
  geom_freqpoly(binwidth = 86400)
```

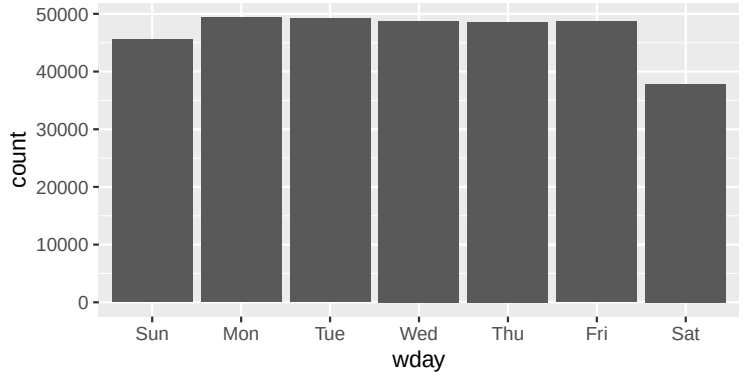


From other types. `as_datetime()`, `as_date()`; numbers are offsets from the Unix epoch 1970-01-01.

8.2.2 Date-time components

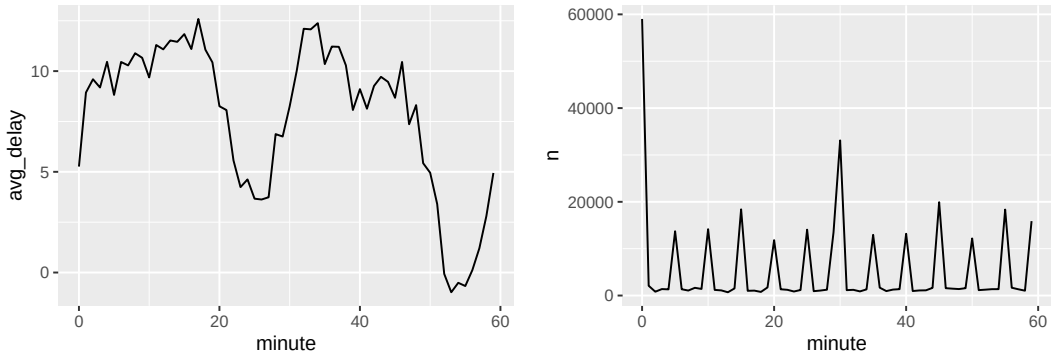
Accessors `year()`, `month()`, `mday()`, `yday()`, `wday()`, `hour()`, `minute()`, `second()`; `label = TRUE` gives names. Fewer flights at weekends:

```
flights_dt %>%
  mutate(wday = wday(dep_time, label = TRUE)) %>%
  ggplot(aes(x = wday)) + geom_bar()
```



By minute of *actual* departure, minutes 20–30 and 50–60 have much lower delays; by *scheduled* minute there is no such pattern: early departures cluster before "nice" scheduled times [Supplement].

```
flights_dt %>%
  mutate(minute = minute(dep_time)) %>%
  group_by(minute) %>%
  summarise(avg_delay = mean(arr_delay, na.rm = TRUE), n = n()) %>%
  ggplot(aes(minute, avg_delay)) + geom_line()
sched_dep <- flights_dt %>%
  mutate(minute = minute(sched_dep_time)) %>%
  group_by(minute) %>%
  summarise(avg_delay = mean(arr_delay, na.rm = TRUE), n = n())
ggplot(sched_dep, aes(minute, n)) + geom_line()
```



Right: scheduled departures prefer round minutes; expect human rounding in human-made data.

Rounding: `floor_date()`, `round_date()`, `ceiling_date()`; flights per week: `count(week = floor_date(dep_time, "week"))`. **Setting**: `year(datetime) <- 2020`, or `update()` for several; overflow rolls over (`mday = 30` in February is in March). `update(dep_time, yday = 1)` shows departures over the hours of one day.

```
datetime <- ymd_hms("2016-07-08 12:34:56")
update(datetime, year = 2020, month = 2, mday = 2, hour = 2)
```

```
## [1] "2020-02-02 02:34:56 UTC"
```

8.2.3 Time spans

Motivation. "Plus one day": 86 400 seconds or the same clock time tomorrow? They differ across daylight saving.

Date subtraction gives a `difftime` (varying unit). Three classes:

- **Durations**: exact seconds, `as.duration(today() - ymd(19791014))`; `dseconds()`, `dminutes()`, `dhours()`, `ddays()`, `dweeks()`, `dyears()`. `ddays(1)` across daylight saving gives an unexpected clock time.
- **Periods**: human units `seconds()`, `days()`, `months()`, `years()`; `days(1)` keeps the clock time, `years(1)` respects leap years. Overnight flights: add `days(1)` when `arr_time < dep_time`.
- **Intervals**: `start %--% end`; divided by a period, the exact number of months etc. (impossible for durations).

```
one_pm <- ymd_hms("2016-03-12 13:00:00", tz = "America/New_York")
one_pm + ddays(1)     # daylight saving started overnight: 14:00
```

```
## [1] "2016-03-13 14:00:00 EDT"
```

```
one_pm + days(1)      # same clock time: 13:00
```

```
## [1] "2016-03-13 13:00:00 EDT"
```

```
next_year <- today() + years(1)
(today() %--% next_year) / days(1)
```

```
## [1] 365
```

Physical time: duration; human time: period; a span in human units: interval.

8.2.4 Time zones

IANA "continent/city" names (`"Asia/Seoul"`); `Sys.timezone()`, `OlsonNames()`. The zone affects only printing: one instant in three zones has differences 0. Default UTC (no daylight saving).

- `with_tz(x, "Asia/Seoul")`: same instant, new display;
- `force_tz(x, "Asia/Seoul")`: same clock reading, new instant (fixes wrong labels).

```
x <- ymd_hms("2024-06-01 12:00:00", tz = "UTC")
with_tz(x, "Asia/Seoul")
```

```
## [1] "2024-06-01 21:00:00 KST"
```

```
force_tz(x, "Asia/Seoul") - x
```

```
## Time difference of -9 hours
```

8.3 Selected exercises

Motivation. Storage versus human reading: level positions, months, ages.

Exercise 8.1 (R4DS Exercise 15.4.1.3)

In the plot of average age by reported income, `fct_relevel(rincome, "Not applicable")` was used. Why did moving "Not applicable" to the front of the levels move it to the bottom of the plot?

Solution. A discrete axis puts the levels at $1, 2, \ldots$ in level order; on the upward y-axis level 1 is at the bottom. `fct_relevel()` makes "Not applicable" level 1 and keeps the others' order, so it drops to the bottom with the brackets in order above (on an x-axis: left). ■

Exercise 8.2 (R4DS Exercise 16.4.5.1)

Why is there `months()` but (in the book's version of lubridate) no `dmonths()`? Explain the outputs below.

```
dmonths(1)
```

```
## [1] "2629800s (~4.35 weeks)"
```

```
ymd("2026-01-31") + months(1)
```

```
## [1] NA
```

```
ymd("2026-01-31") %m+% months(1)
```

```
## [1] "2026-02-28"
```

```
ymd("2026-01-31") + dmonths(1)
```

```
## [1] "2026-03-02 10:30:00 UTC"
```

Solution.

Step 1. Months have 28–31 days, so no fixed number of seconds is "one month"; a period ("same day next month") needs no fixed length. Hence only `months()`.

Step 2. Newer `dmonths(1)` is the average month, $365.25/12 = 30.4375$ days $= 2\,629\,800$ s; 31 January + that is 10:30 on 2 March.

Step 3. `months(1)` asks for 31 February: `NA`, not a silent guess. `%m+%` rolls back to 28 February. The right answer depends on the application. ■

Exercise 8.3 (an age that is right on the birthday; after R4DS Exercise 16.4.5.4)

(a) Write a function that returns the age in completed years of a person born on `bday` at date `on`.
(b) Someone born on 31 October 2000 turns 26 on 31 October 2026. Show that dividing the duration of the span by `dyears(1)` gives a number below 26 on that day, while dividing the interval by `years(1)` gives exactly 26.

```
age <- function(bday, on = today()) floor((bday %--% on) / years(1))
bday <- ymd("2000-10-31")
age(bday, on = ymd("2026-10-31"))
```

```
## [1] 26
```

```
age(bday, on = ymd("2026-10-30"))
```

```
## [1] 25
```

```
as.duration(bday %--% ymd("2026-10-31")) / dyears(1)
```

```
## [1] 25.99863
```

Solution [Supplement].

(a) Interval / `years(1)` counts calendar years (fraction for the rest); `floor()` keeps completed years, so `age()` steps up exactly on each birthday.

(b)

Step 1. The span has 26 years and six leap days (29 Feb 2004, . . . , 2024): $26 \cdot 365 + 6 = 9496$ days.

Step 2. `dyears(1)` $= 365.25$ days and $26 \cdot 365.25 = 9496.5$, so $9496/365.25 = 25.9986 < 26$ (6 leap days, not 6.5); `floor()` would give 25.

Step 3. The period counts calendar years, regardless of leap days: exactly 26. Ages need periods. ■